



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 10

NOMBRE COMPLETO: Lechuga Castillo Shareny Ixchel

N° de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 05-11-2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Ejercicio 1: Animar algún objeto por medio de Keyframes (que no sea ninguno de los personajes principales ni humanoide) requiere compartir capturas de pantalla donde se vea la impresión de los keyframes guardados o reproducidos en consola y guardados en un archivo de tipo txt y compartir los keyframes para que yo pueda reproducir su animación. La única restricción es que la Animación debe de representar el movimiento de un cuerpo en la vida real lógico, por ejemplo: una caja que va rodando no es un movimiento lógico. Un helado flotando no es un movimiento lógico.

El código que obtenemos es el siguiente:

```
Camera camera;

Texture brickTexture;
Texture dirtTexture;
Texture plainTexture;
Texture pisoTexture;
Texture AgaveTexture;
Texture FlechaTexture;

Model Kitt_M;
Model Llanta_M;
Model Blackhawk_M;
Model Nave;

Skybox skybox;

/////////////////////////////////KEYFRAMES/////////////////////////////////
|
bool animacion = false;

//NEW// Keyframes
float posXNave = -35.0, posYNave = 10.0, posZNave = -35.0;
float movNave_x = 0.0f, movNave_y = 0.0f;
float giroNave = 0;

#define MAX_FRAMES 100
int i_max_steps = 900;
int i_curr_steps = 5;

typedef struct _frame
{
    //Variables para GUARDAR Key Frames
    float movNave_x;        //Variable para PosicionX
    float movNave_y;        //Variable para PosicionY
    float movNave_xInc;     //Variable para IncrementoX
    float movNave_yInc;     //Variable para IncrementoY
    float giroNave;         //Variable para Rotación
    float giroNaveInc;      //Variable para Incremento de Rotación
} FRAME;

FRAME KeyFrame[MAX_FRAMES];
int FrameIndex = 5;        //introducir datos
bool play = false;
int playIndex = 0;

bool primerosGuardados = false; // Verifica si los primeros 7 keyframes ya fueron guardados
bool animacionIniciada = false; // Verifica si la animación fue iniciada con SPACE

void GuardarPrimerosKeyFrames()
{
    std::ofstream file("salvadodeKeyframes.txt");
    if (!file) {
        printf("Error al abrir el archivo salvadodeKeyframes.txt \n");
        return;
    }
}
```

```

file << "Primeros Keyframes:\n";
for (int i = 0; i < FrameIndex; i++) { // Guardamos los primeros 7
    file << "Keyframe [" << i << "]:\n";
    file << "movNave_x: " << KeyFrame[i].movNave_x << "\n";
    file << "movNave_y: " << KeyFrame[i].movNave_y << "\n";
    file << "giroNave: " << KeyFrame[i].giroNave << "\n";
    file << "-----\n";
}
file.close();
printf("Los primeros 5 Keyframes han sido guardados en salvadodeKeyframes.txt \n");

primerosGuardados = true; // Marca como guardado
}

void GuardarKeyFrame(int frameIndex)
{
    std::ofstream file("salvadodeKeyframes.txt", std::ios::app); // Modo append para agregar al archivo
    if (!file) {
        printf("Error al abrir el archivo salvadodeKeyframes.txt \n");
        return;
    }

    file << "Keyframe [" << frameIndex << "]:\n";
    file << "movNave_x: " << KeyFrame[frameIndex].movNave_x << "\n";
    file << "movNave_y: " << KeyFrame[frameIndex].movNave_y << "\n";
    file << "giroNave: " << KeyFrame[frameIndex].giroNave << "\n";
    file << "-----\n";
    file.close();

    printf("Keyframe [%d] guardado en salvadodeKeyframes.txt \n", frameIndex);
}

void saveFrame()
{
    printf("frameindex %d\n", FrameIndex);

    KeyFrame[frameIndex].movNave_x = movNave_x;
    KeyFrame[frameIndex].movNave_y = movNave_y;
    KeyFrame[frameIndex].giroNave = giroNave;

    GuardarKeyFrame(FrameIndex); // Llama a la función para guardar el frame actual en el archivo

    FrameIndex++;

    printf("Keyframe guardado en archivo.\n");
}

void resetElements(void) //Tecla 0
{
    movNave_x = KeyFrame[0].movNave_x;
    movNave_y = KeyFrame[0].movNave_y;
    giroNave = KeyFrame[0].giroNave;
}

void interpolation(void)
{
    KeyFrame[playIndex].movNave_xInc = (KeyFrame[playIndex + 1].movNave_x - KeyFrame[playIndex].movNave_x) / i_max_steps;
    KeyFrame[playIndex].movNave_yInc = (KeyFrame[playIndex + 1].movNave_y - KeyFrame[playIndex].movNave_y) / i_max_steps;
    KeyFrame[playIndex].giroNaveInc = (KeyFrame[playIndex + 1].giroNave - KeyFrame[playIndex].giroNave) / i_max_steps;
}

void animate(void)
{
    //Movimiento del objeto con barra espaciadora
    if (play)
    {
        if (i_curr_steps >= i_max_steps) //fin de animación entre frames?
        {
            playIndex++;
            printf("playindex : %d\n", playIndex);
            if (playIndex > FrameIndex - 2) //Fin de toda la animación con último frame?
            {
                printf("Frame index= %d\n", FrameIndex);
                printf("termino la animacion\n");
                playIndex = 0;
                play = false;
            }
            else //Interpolación del próximo cuadro
            {
                i_curr_steps = 0; //Resetea contador
                interpolation();
            }
        }
        else
        {
            //Dibujar Animación
            movNave_x += KeyFrame[playIndex].movNave_xInc;
            movNave_y += KeyFrame[playIndex].movNave_yInc;
            giroNave += KeyFrame[playIndex].giroNaveInc;
            i_curr_steps++;
        }
    }
}

```

```

Kitt_M = Model();
Kitt_M.LoadModel("Models/kitt_optimizado.obj");
Llanta_M = Model();
Llanta_M.LoadModel("Models/llanta_optimizada.obj");
Blackhawk_M = Model();
Blackhawk_M.LoadModel("Models/uh60.obj");
Nave = Model();
Nave.LoadModel("Models/Nave.obj");

//-----PARA TENER KEYFRAMES GUARDADOS NO VOLATILES QUE SIEMPRE SE UTILIZARAN SE DECLARAN AQUÍ

// Definimos los keyframes de la animación
KeyFrame[0].movNave_x = 0.0f;
KeyFrame[0].movNave_y = 0.0f;
KeyFrame[0].giroNave = 0.0f;

KeyFrame[1].movNave_x = 70.0f;
KeyFrame[1].movNave_y = 00.0f;
KeyFrame[1].giroNave = 0.0f;

KeyFrame[2].movNave_x = 70.0f;
KeyFrame[2].movNave_y = 00.0f;
KeyFrame[2].giroNave = 180.0f;

KeyFrame[3].movNave_x = 0.0f;
KeyFrame[3].movNave_y = 00.0f;
KeyFrame[3].giroNave = 180.0f;

KeyFrame[4].movNave_x = 0.0f;
KeyFrame[4].movNave_y = 00.0f;
KeyFrame[4].giroNave = 0.0f;

//Se agregan nuevos frames

printf("\nTeclas para uso de Keyframes:\n1.-Presionar barra espaciadora para reproducir animacion.\n2.-Presionar 0 para volver a habilitar reproduccion de la animacion\n");
printf("3.-Presiona L para guardar frame\n4.-Presiona P para habilitar guardar nuevo frame\n5.-Presiona 1 para mover en X\n6.-Presiona 2 para habilitar mover en X");

////Loop mientras no se cierra la ventana
while (!mainWindow.getShouldClose())
{
    GLfloat now = glfwGetTime();
    deltaTime = now - lastTime;
    deltaTime += (now - lastTime) / limitFPS;
    lastTime = now;

    angulovaria += 0.5f * deltaTime;

    if (movCoche < 30.0f)
    {
        movCoche -= movOffset * deltaTime;
        //printf("avanza%f \n ",movCoche);
    }
    rotllanta += rotllantaOffset * deltaTime;

    //Recibir eventos del usuario
    glfwPollEvents();
    camera.keyControl(mainWindow.getKeys(), deltaTime);
    camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());

    //-----Para Keyframes
    inputKeyframes(mainWindow.getKeys());
    animate();
}

```

```

glm::mat4 model(1.0);
glm::mat4 modelaux(1.0);
glm::vec3 color = glm::vec3(1.0f, 1.0f, 1.0f);
glm::vec2 toffset = glm::vec2(0.0f, 0.0f);

model = glm::mat4(1.0);
model = glm::scale(model, glm::vec3(4.0f, 4.0f, 4.0f));
model = modelaux;
model = glm::translate(model, glm::vec3(35.0f, -11.0f, 36.0f));

posblackhawk = glm::vec3(posXnave + movNave_x, posYnave + movNave_y, posZnave);
model = glm::translate(model, posblackhawk);
model = glm::rotate(model, giroNave * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
//model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
//color = glm::vec3(0.0f, 1.0f, 0.0f);
//glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Nave.RenderModel();

glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
pisoTexture.UseTexture();
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);

meshList[2]->RenderMesh();

toffset = glm::vec2(0.0, 0.0);
glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
//Agave ¿qué sucede si lo renderizan antes del coche y de la pista?

```

```

void inputKeyframes(bool* keys)
{
    if (keys[GLFW_KEY_SPACE])
    {
        if (reproduciranimacion < 1)
        {
            if (play == false && (FrameIndex > 1))
            {
                resetElements();
                // First Interpolation
                interpolation();
                play = true;
                playIndex = 0;
                i_curr_steps = 0;
                reproduciranimacion++;
                printf("\n presiona 0 para habilitar reproducir de nuevo la animación\n");
                habilitaranimacion = 0;
                animacionIniciada = true; // Marca que la animación ha sido iniciada
            }
            else
            {
                play = false;
            }
        }
    }
    if (keys[GLFW_KEY_0])
    {
        if (habilitaranimacion < 1 && reproduciranimacion > 0)
        {
            printf("Ya puedes reproducir de nuevo la animación con la tecla de barra espaciadora\n");
            reproduciranimacion = 0;
            habilitaranimacion++;
        }
    }
}

```

```

if (keys[GLFW_KEY_L])
{
    if (animacionIniciada && !primerosGuardados) {
        // Si la animación ha sido iniciada con SPACE y los primeros 5 keyframes no se han guardado
        GuardarPrimerosKeyFrames();
    }

    if (guardoFrame < 1)
    {
        saveFrame();
        printf("movNave_x es: %f\n", movNave_x);
        printf("movNave_y es: %f\n", movNave_y);
        printf("presiona P para habilitar guardar otro frame'\n");
        guardoFrame++;
        reinicioFrame = 0;
    }
}

if (keys[GLFW_KEY_P])
{
    if (reinicioFrame < 1)
    {
        guardoFrame = 0;
        reinicioFrame++;
        printf("Ya puedes guardar otro frame presionando la tecla L'\n");
    }
}

if (keys[GLFW_KEY_1])
{
    if (ciclo < 1)
    {
        movNave_x -= 1.0f;
        printf("\n movNave_x es: %f\n", movNave_x);
        ciclo++;
        ciclo2 = 0;
        printf("\n Presiona la tecla 2 para poder habilitar la variable\n");
    }
}

if (keys[GLFW_KEY_2])
{
    if (ciclo2 < 1)
    {
        ciclo = 0;
        ciclo2++;
        printf("\n Ya puedes modificar tu variable movNave_x presionando la tecla 1\n");
    }
}

if (keys[GLFW_KEY_3])
{
    if (ciclo < 1)
    {
        movNave_x += 1.0f;
        printf("\n movNave_x es: %f\n", movNave_x);
        ciclo++;
        ciclo2 = 0;
        printf("\n Presiona la tecla 4 para poder habilitar la variable\n");
    }
}

if (keys[GLFW_KEY_4])
{
    if (ciclo2 < 1)
    {
        ciclo = 0;
        ciclo2++;
        printf("\n Ya puedes modificar tu variable movNave_x presionando la tecla 3\n");
    }
}

if (keys[GLFW_KEY_5])
{
    if (ciclo < 1)
    {
        movNave_y -= 1.0f;
        printf("\n movNave_y es: %f\n", movNave_y);
        ciclo++;
        ciclo2 = 0;
        printf("\n Presiona la tecla 6 para poder habilitar la variable\n");
    }
}

if (keys[GLFW_KEY_6])
{
    if (ciclo2 < 1)
    {
        ciclo = 0;
        ciclo2++;
        printf("\n Ya puedes modificar tu variable movNave_y presionando la tecla 5\n");
    }
}

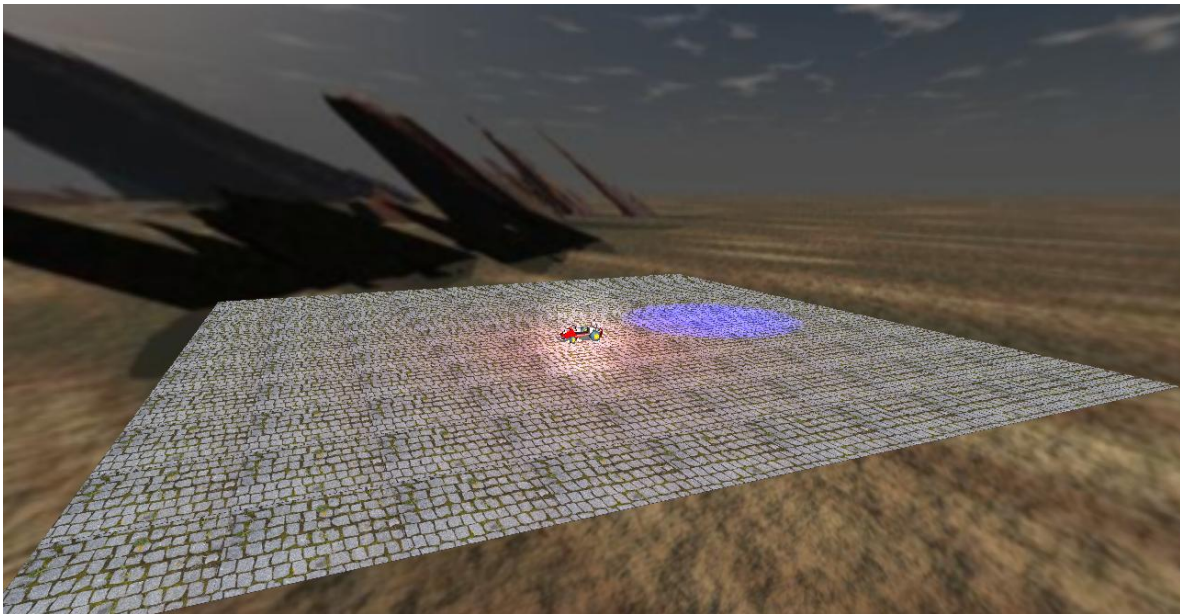
```

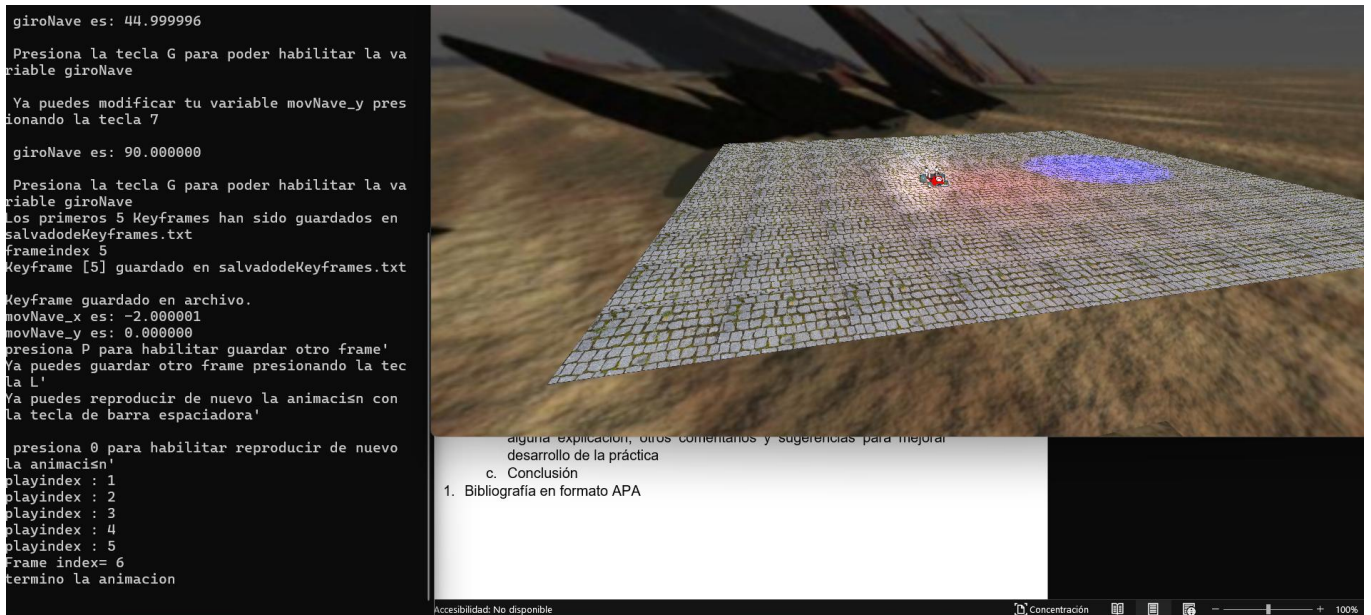
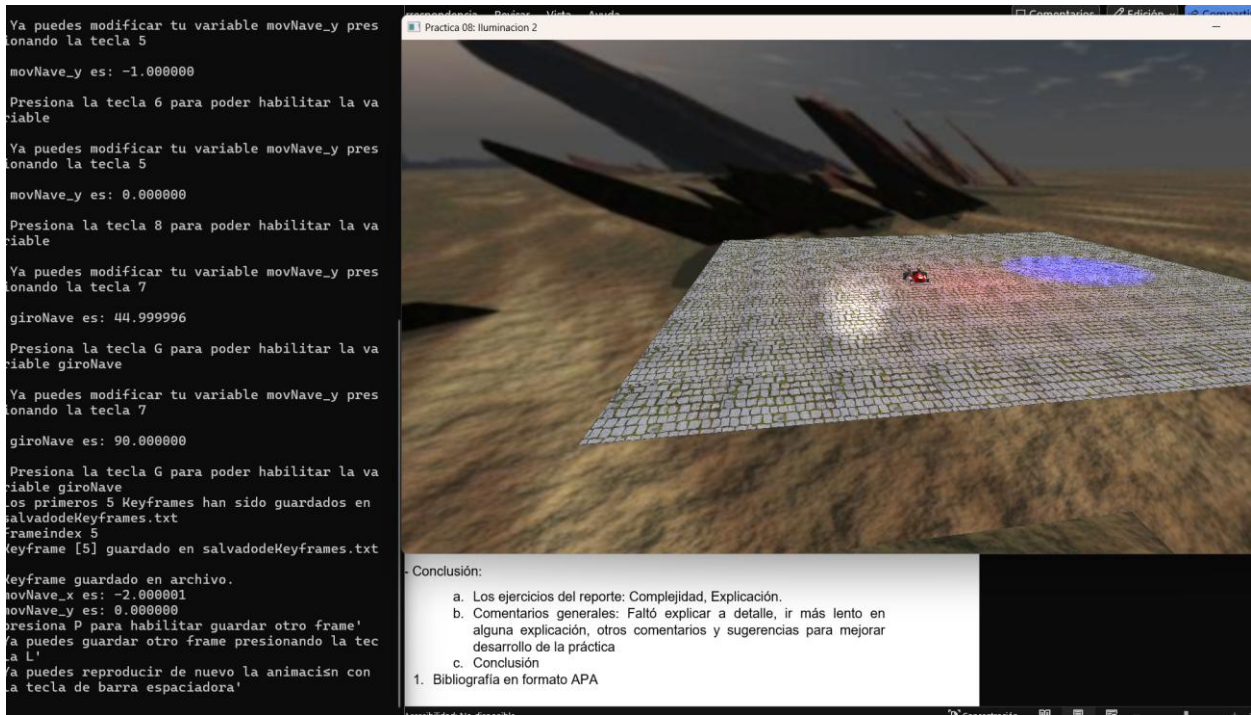
```

if (keys[GLFW_KEY_7])
{
    if (ciclo < 1)
    {
        movNave_y += 1.0f;
        printf("\n movNave_y es: %f\n", movNave_y);
        ciclo++;
        ciclo2 = 0;
        printf("\n Presiona la tecla 8 para poder habilitar la variable\n");
    }
}
if (keys[GLFW_KEY_8])
{
    if (ciclo2 < 1)
    {
        ciclo = 0;
        ciclo2++;
        printf("\n Ya puedes modificar tu variable movNave_y presionando la tecla 7\n");
    }
}

if (keys[GLFW_KEY_9])
{
    if (ciclo < 1)
    {
        giroNave += 45.0f;
        printf("\n giroNave es: %f\n", giroNave);
        ciclo++;
        ciclo2 = 0;
        printf("\n Presiona la tecla G para poder habilitar la variable giroNave\n");
    }
}
if (keys[GLFW_KEY_G])
{
    if (ciclo2 < 1)
    {
        ciclo = 0;
        ciclo2++;
        printf("\n Ya puedes modificar tu variable de giroNave presionando la tecla 9\n");
    }
}

```





Problemas:

Durante esta práctica no surgió ningún problema, pues solo se aplicaron conceptos ya realizados en otras prácticas anteriores, y únicamente fue necesario analizar los casos y plasmarlos correctamente.

Conclusión:

Esta práctica fue una de las más sencillas que he realizado, y gracias a ella pude entender de una mejor manera cómo funciona la animación por keyframe.

Al principio este tipo de animaciones me parecía sumamente complicado, pero gracias a esta práctica logré comprender mejor su funcionamiento y lo sencillo que puede llegar a ser.